SEP2/ In-class assignment 1 (Individual): Localization Console

AD/ SPRINT 1
Fall 2026

Note: In-class assignments are compulsory for completing both the sprint implementation and the overall project. Failure to complete in-class assignments will result in forfeiting the implementation portion of your sprint tasks and will impact your overall course grade. To pass the course, you must achieve a total score above the 2-point threshold.

Objective

Implement the TemperatureConverter class, write JUnit 5 unit tests, generate and analyze a JaCoCo code coverage report, and build an automated CI/CD pipeline using Docker.

Tasks & Instructions

Step 1: Create the TemperatureConverter Class

- Implement fahrenheitToCelsius(double fahrenheit): Returns (fahrenheit - 32) * 5 / 9.
- Implement celsiusToFahrenheit(double celsius): Returns (celsius * 9 / 5) + 32.
- Implement isExtremeTemperature(double celsius): Returns true if temperature is below -40°C or above 50°C.

Step 2: Localization (i18n)

- Prompt the user to select their preferred language at application launch:
  - *English*
  - *Finnish*
  - *Swedish*
  - *Japanese*
- Create the corresponding UTF-8 properties files:
  - *MessagesBundle_en_US.properties (English — Default)*
  - *MessagesBundle_fi_FI.properties (Finnish)*
  - *MessagesBundle_sv_SE.properties (Swedish)*
  - *MessagesBundle_ja_JP.properties (Japanese)*

Step 3: Write Unit Tests

- Create unit tests using JUnit 5 in TemperatureConverterTest.
- Test fahrenheitToCelsius with multiple inputs.
- Test celsiusToFahrenheit with multiple inputs.
- Test isExtremeTemperature with boundary and edge cases (e.g., -40, 50, -41, 51).

Step 4: CI/CD Pipeline & Dockerization

- Create a Dockerfile to package and run your application.
- Configure a CI/CD pipeline (e.g., Jenkins or GitHub Actions) to:
  - *Build and run unit tests.*
  - *Generate a JaCoCo code coverage report.*
  - *Build and push the Docker image to Docker Hub.*

Focus Points

- UTF-8 Encoding: Ensure all .properties resource bundles use UTF-8.
- Code Coverage: Aim for high branch/line coverage across all methods in TemperatureConverter.
- CI/CD & Containerization: Demonstrate clear automation from test execution to Docker image deployment.

Submission Requirements

Submit the following items via Oma:

1. JaCoCo Coverage Screenshot: A screenshot displaying successful unit test execution and the JaCoCo code coverage report.
2. Localization Execution Screenshot: A screenshot demonstrating successful execution of the localized application in a non-default language (e.g., Japanese).
3. Repository Link: A link to your public GitHub repository containing the source code, tests, Dockerfile, and CI/CD configuration.